LAPORAN PRAKTIKUM

MATEMATIKA DISKRIT LANJUT

MODUL 1: PENGENALAN GRAF DAN VISUALISASI DASAR

Laporan Ini Disusun Untuk Memenuhi Tugas Praktikum
Mata Kuliah : Matematika Diskrit Lanjut



Disusun Oleh :
Nama : Ghaisan Khoirul Badruzaman
NIM : 251524048
Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

BAB I PENDAHULUAN

Modul 1 praktikum Matematika Diskrit Lanjut mengenalkan graf lewat pemrograman. Graf dibangun dan divisualisasikan dengan library NetworkX dan Matplotlib di Python, mulai dari lima jenis graf dasar sampai graf dari contoh model di buku Rosen, yaitu jaringan komputer, graf pertemanan, jaringan semantik, dan turnamen.

Modul menyarankan Thonny sebagai IDE. Pengerjaan kali ini dilakukan di MacBook, jadi program dijalankan langsung dengan Python dari terminal. Versi Python dan library yang dipakai dicetak di Task01. Kodenya tetap bisa dibuka dan dijalankan di Thonny tanpa perubahan.

Tabel 1.1: Isi arsip pengumpulan

Berkas	**Isi**
`[MDL2026_2B_D4_2025]_Modul1_048/`	`Task01_Hello Python.py` dan `Task02_Hello Networkx.py`
`[MDL2026_2B_D4_2025]_Modul1_048.pdf`	Laporan praktikum ini
`[MDL2026_2B_D4_2025]_Modul1_048.docx`	Laporan praktikum ini dalam format Word
`[MDL2026_2B_D4_2025]_Modul1_048.txt`	Dokumentasi referensi resmi dan tanya jawab selama pengerjaan

BAB II TASK01_HELLO PYTHON

2.1 Kode Program

```
import sys

import matplotlib
import networkx as nx

print("Hello, Python!")
print("Hello, Matematika Diskrit Lanjut!")
print()

# cek versi supaya tahu library untuk praktikum sudah terpasang
print(f"Versi Python     : {sys.version.split()[0]}")
print(f"Versi NetworkX   : {nx.__version__}")
print(f"Versi Matplotlib : {matplotlib.__version__}")
```

2.2 Output Program

```
Hello, Python!
Hello, Matematika Diskrit Lanjut!

Versi Python     : 3.14.6
Versi NetworkX   : 3.6.1
Versi Matplotlib : 3.10.8
```

2.3 Lesson Learnt

Task ini sederhana, tapi berguna untuk memastikan NetworkX dan Matplotlib benar-benar terpasang di Python yang dipakai. Di Mac biasanya ada lebih dari satu Python, misalnya bawaan sistem dan yang dipasang lewat Homebrew, jadi mencetak versi library dari dalam program lebih meyakinkan daripada sekadar melihat hasil `pip install`.

BAB III TASK02_HELLO NETWORKX

3.1 Jenis-jenis Graf

Setiap jenis graf pada modul punya kelas yang sesuai di NetworkX. Kelas yang dipilih menentukan apakah sisi ganda dan arah sisi ikut disimpan.

Tabel 3.1: Jenis graf dan kelas NetworkX yang dipakai

Jenis graf	Sisi	Sisi ganda	Gelang	Kelas NetworkX
Graf sederhana	Tidak berarah	Tidak ada	Tidak ada	`nx.Graph()`
Graf ganda	Tidak berarah	Ada	Tidak ada	`nx.MultiGraph()`
Graf semu	Tidak berarah	Ada	Ada	`nx.MultiGraph()`
Graf berarah	Berarah	Tidak ada	Ada	`nx.DiGraph()`
Graf ganda berarah	Berarah	Ada	Ada	`nx.MultiDiGraph()`

3.2 Latihan Praktikum

Latihan meminta lima graf pada Gambar 12 modul dibuat ulang. Himpunan sisi tiap graf ditulis sesuai gambar, lalu kelimanya digambar dalam satu jendela supaya mudah dibandingkan.

```
G1 = nx.Graph()
G1.add_edges_from([("a", "b"), ("a", "c"), ("b", "c"), ("c", "d"), ("d", "a")])

# e4 dan e5 sama-sama menghubungkan c-d, e6 dan e7 sama-sama a-d
G2 = nx.MultiGraph()
G2.add_edges_from([("a", "b"), ("b", "c"), ("a", "c"), ("c", "d"), ("c", "d"), ("a", "d"),
    ("a", "d")])

# graf semu = graf ganda G2 ditambah gelang e8 di simpul b
G3 = nx.MultiGraph(G2)
G3.add_edge("b", "b")

G4 = nx.DiGraph()
G4.add_edges_from([("a", "b"), ("b", "c"), ("a", "c"), ("d", "c"), ("d", "a")])

G5 = nx.MultiDiGraph()
G5.add_edges_from([("a", "b"), ("b", "c"), ("a", "c"), ("d", "c"), ("d", "c"),
                   ("a", "d"), ("d", "a"), ("b", "b")])
```

Fungsi `analisis()` mencetak jumlah simpul, jumlah sisi, dan derajat tiap simpul. Untuk graf berarah yang dicetak derajat masuk dan derajat keluar. Sisi ganda digambar melengkung lewat `connectionstyle`:

```
if G.is_multigraph():
    # sisi ke-k antara dua simpul yang sama dibuat melengkung beda-beda supaya tidak menumpuk
    rad = [0.1, 0.35, -0.2, 0.55] if G.is_directed() else [0.0, 0.18, -0.18, 0.35]
    nx.draw_networkx_edges(G, pos, ax=ax, node_size=ukuran, arrows=True, arrowsize=18,
                           arrowstyle="-|>" if G.is_directed() else "-",
                           connectionstyle=[f"arc3,rad={r}" for r in rad])
```

Hasil eksekusi bagian latihan ditampilkan berikut ini.

```
===== LATIHAN: JENIS-JENIS GRAF (GAMBAR 12) =====
```

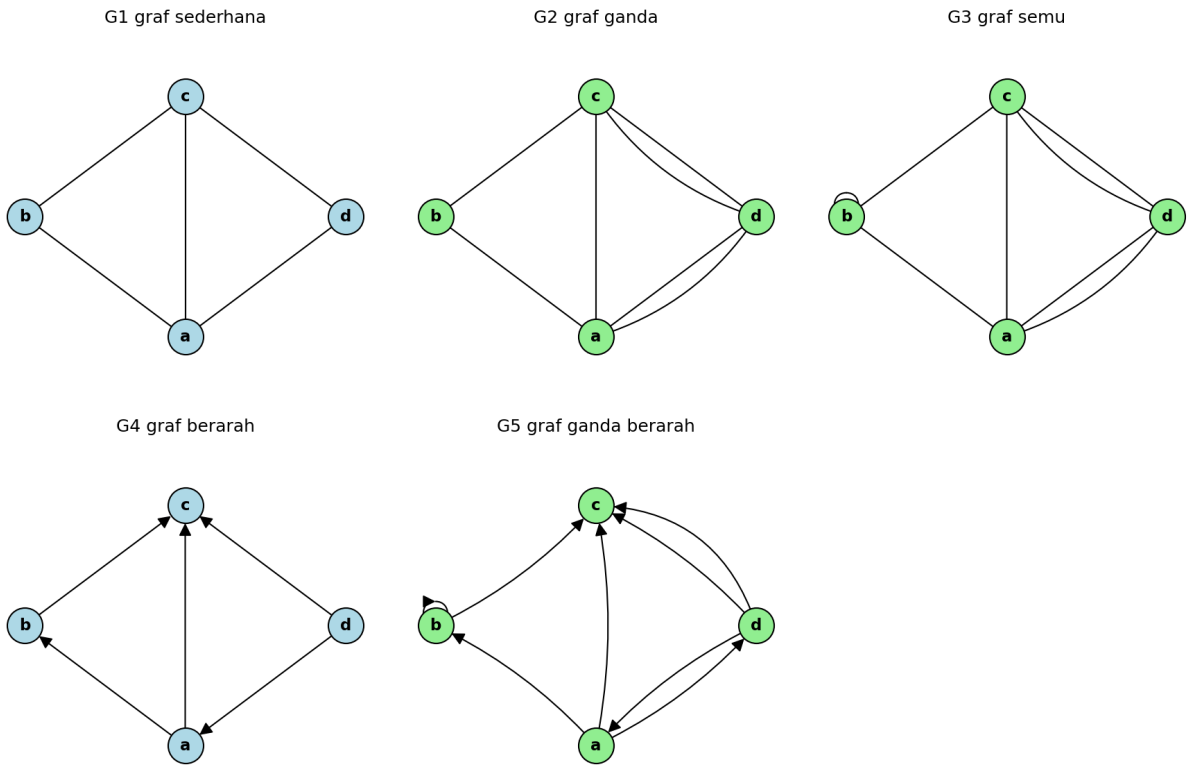
```
--- Analisis G1 graf sederhana ---
Jumlah simpul : 4
Jumlah sisi   : 5
Derajat : a = 3, b = 2, c = 3, d = 2

--- Analisis G2 graf ganda ---
Jumlah simpul : 4
Jumlah sisi   : 7
Derajat : a = 4, b = 2, c = 4, d = 4

--- Analisis G3 graf semu ---
Jumlah simpul : 4
Jumlah sisi   : 8
Derajat : a = 4, b = 4, c = 4, d = 4

--- Analisis G4 graf berarah ---
Jumlah simpul : 4
Jumlah sisi   : 5
Derajat : a (masuk 1, keluar 2), b (masuk 1, keluar 1), c (masuk 3, keluar 0), d (masuk 0,
          keluar 2)

--- Analisis G5 graf ganda berarah ---
Jumlah simpul : 4
Jumlah sisi   : 8
Derajat : a (masuk 1, keluar 3), b (masuk 2, keluar 2), c (masuk 4, keluar 0), d (masuk 1,
          keluar 3)
```



Gambar 3.1: Visualisasi graf G1 sampai G5

Derajat hasil program cocok dengan hitungan manual dari gambar. Pada G3, simpul b berderajat 4 walaupun hanya bertetangga dengan a dan c, karena gelang e8 dihitung dua kali. Jumlah derajat

G2 adalah 14 dan G3 adalah 16, tepat dua kali jumlah sisinya.

3.3 Lesson Learnt Latihan

Contoh kode di modul ternyata tidak persis sama dengan gambarnya. Kode G2 menambahkan sisi `('a', 'd')` tiga kali sehingga jumlah sisinya 8, padahal f(e6) = f(e7) = {a, d} berarti hanya dua sisi dan totalnya 7. Kode G3 juga tidak memuat sisi a-c (e3), dan gelangnya dipasang di c, bukan di b seperti f(e8) = {b}. Karena itu himpunan sisi di Task02 ditulis ulang langsung dari definisi dan gambar, bukan menyalin contoh.

Pemilihan kelas graf juga menentukan hasil. Kalau G2 dibuat dengan `nx.Graph()`, sisi ganda c-d dan a-d diam-diam digabung menjadi satu sehingga jumlah sisinya tinggal 5, tanpa pesan error sama sekali.

Menggambar sisi ganda juga butuh perhatian. Tanpa `connectionstyle`, sisi ganda tergambar sebagai garis lurus yang saling menumpuk dan terlihat seperti satu sisi. Pada graf berarah, lengkungan kecil dipakai untuk semua sisi supaya a->d dan d->a tidak tertumpuk.

3.4 Tugas Praktikum

Tugas meminta enam graf pada gambar modul dibuat seperti latihan. Keenamnya adalah contoh model graf dari buku Rosen. Ringkasan hasilnya ada di Tabel 3.2.

Tabel 3.2: Ringkasan graf tugas praktikum

Gambar	Model	Jenis graf	Simpul	Sisi
Figure 1	Jaringan komputer antar data center	Graf sederhana	7	9
Figure 2	Jaringan dengan jalur ganda	Graf ganda	7	13
Figure 3	Jaringan dengan diagnostic link	Graf semu	7	20
Figure 6	Graf pertemanan	Graf sederhana	16	25
Figure 13	Jaringan semantik kata "mouse"	Graf sederhana	20	28
Figure 15	Turnamen sistem gugur	Graf sederhana berbentuk pohon	15	14

Figure 1 sampai 3 memakai simpul yang sama. Figure 2 menambahkan jalur kedua pada empat pasang kota, lalu Figure 3 menambahkan satu gelang di setiap data center:

```
jalur = [("San Francisco", "Los Angeles"), ("San Francisco", "Denver"), ("Los Angeles",
    "Denver"),
         ("Denver", "Chicago"), ("Chicago", "Detroit"), ("Chicago", "New York"), ("Chicago",
             "Washington"),
         ("Detroit", "New York"), ("New York", "Washington")]

F1 = nx.Graph()
F1.add_edges_from(jalur)

# jalur ganda: LA-Denver, Denver-Chicago, Chicago-New York, New York-Washington
F2 = nx.MultiGraph()
```

```
F2.add_edges_from(jalur)
F2.add_edges_from([("Los Angeles", "Denver"), ("Denver", "Chicago"), ("Chicago", "New York"),
                   ("New York", "Washington")])

# diagnostic link = gelang di setiap data center
F3 = nx.MultiGraph(F2)
F3.add_edges_from((k, k) for k in kota)
```

Pada Figure 15, setiap simpul adalah satu pertandingan dan sisi menghubungkan pertandingan dengan pertandingan berikutnya yang diikuti pemenangnya:

```
lanjut = [("Stanford vs Georgia", "Stanford vs Xavier"), ("Xavier vs Gonzaga", "Stanford vs
    Xavier"),
          ("Oklahoma vs Notre Dame", "Oklahoma vs Kentucky"), ("Kentucky vs Nebraska",
              "Oklahoma vs Kentucky"),
          ("Stanford vs Xavier", "Stanford vs Oklahoma"), ("Oklahoma vs Kentucky", "Stanford
              vs Oklahoma"),
          ("Stanford vs Oklahoma", "Connecticut vs Stanford"), ("Connecticut vs Baylor",
              "Connecticut vs Stanford"),
          ("Connecticut vs Florida State", "Connecticut vs Baylor"), ("Baylor vs Duke",
              "Connecticut vs Baylor"),
          ("Connecticut vs Iowa State", "Connecticut vs Florida State"),
          ("Florida State vs Mississippi State", "Connecticut vs Florida State"),
          ("Baylor vs Tennessee", "Baylor vs Duke"), ("Duke vs San Diego State", "Baylor vs
              Duke")]
F15 = nx.Graph()
F15.add_edges_from(lanjut)
analisis(F15, "Figure 15 turnamen sistem gugur")
print(f"Apakah graf berbentuk pohon (tree)? {nx.is_tree(F15)}")
# nama sebelum "vs" adalah pemenang, jadi juara diambil dari pertandingan final
final = "Connecticut vs Stanford"
print(f"Pertandingan final: {final}, derajat {F15.degree(final)}")
print(f"Juara turnamen: {final.split(' vs ')[0]}\n")
```

Daftar sisi Figure 6 dan Figure 13 ditulis dengan pola yang sama, lengkapnya ada di berkas `Task02_Hello Networkx.py`. Hasil eksekusi bagian tugas ditampilkan berikut ini.

```
===== TUGAS PRAKTIKUM =====

--- Analisis Figure 1 jaringan komputer (graf sederhana) ---
Jumlah simpul : 7
Jumlah sisi   : 9
Derajat : San Francisco = 2, Los Angeles = 2, Denver = 3, Chicago = 4, Detroit = 2, New
          York = 3, Washington = 2

--- Analisis Figure 2 jalur ganda antar data center (graf ganda) ---
Jumlah simpul : 7
Jumlah sisi   : 13
Derajat : San Francisco = 2, Los Angeles = 3, Denver = 5, Chicago = 6, Detroit = 2, New
          York = 5, Washington = 3

--- Analisis Figure 3 dengan diagnostic link (graf semu) ---
Jumlah simpul : 7
Jumlah sisi   : 20
Derajat : San Francisco = 4, Los Angeles = 5, Denver = 7, Chicago = 8, Detroit = 4, New
          York = 7, Washington = 5

--- Analisis Figure 6 graf pertemanan ---
Jumlah simpul : 16
Jumlah sisi   : 25
Derajat : Jan = 4, Kamini = 1, Paula = 7, Lila = 5, Joel = 5, Eduardo = 1, Todd = 4, Liz =
          4, Amy = 4, Steve = 5, Koko = 1, Kamlesh = 2, Ching = 1, Gail = 3, Shaquira = 2,
          Kari = 1

--- Analisis Figure 13 jaringan semantik kata mouse ---
```

```
Jumlah simpul : 20
Jumlah sisi   : 28
Derajat : Wolf = 1, Fox = 3, Dog = 4, Rabbit = 3, Cat = 2, Rat = 3, Mouse = 6, Desktop =
          5, Keyboard = 4, Trackpad = 2, Lizard = 4, Duck = 1, Turtle = 1, Fly = 3, Spider
          = 1, Cockroach = 1, Printer = 3, Laptop = 5, Router = 3, Tablet = 1

--- Analisis Figure 15 turnamen sistem gugur ---
Jumlah simpul : 15
Jumlah sisi   : 14
Derajat : Stanford vs Georgia = 1, Stanford vs Xavier = 3, Xavier vs Gonzaga = 1, Oklahoma
          vs Notre Dame = 1, Oklahoma vs Kentucky = 3, Kentucky vs Nebraska = 1, Stanford
          vs Oklahoma = 3, Connecticut vs Stanford = 2, Connecticut vs Baylor = 3,
          Connecticut vs Florida State = 3, Baylor vs Duke = 3, Connecticut vs Iowa State
          = 1, Florida State vs Mississippi State = 1, Baylor vs Tennessee = 1, Duke vs
          San Diego State = 1

Apakah graf berbentuk pohon (tree)? True
Pertandingan final: Connecticut vs Stanford, derajat 2
Juara turnamen: Connecticut
```

Figure 1 jaringan komputer (graf sederhana)

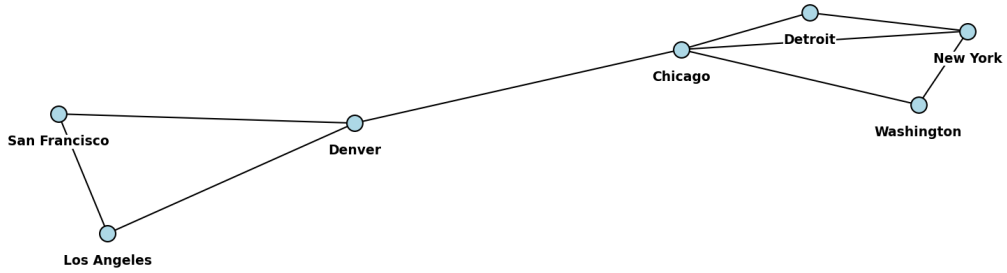


Figure 2 jalur ganda antar data center (graf ganda)

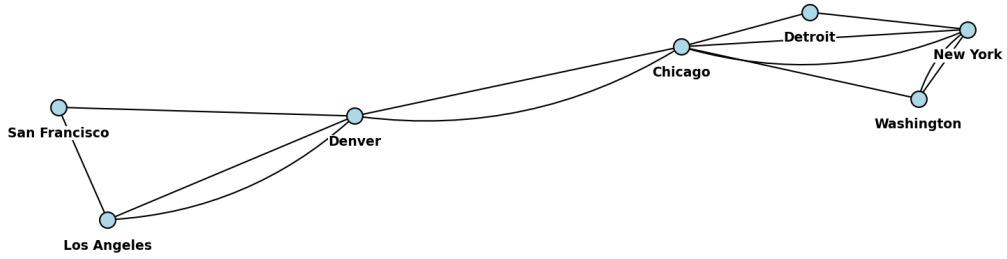
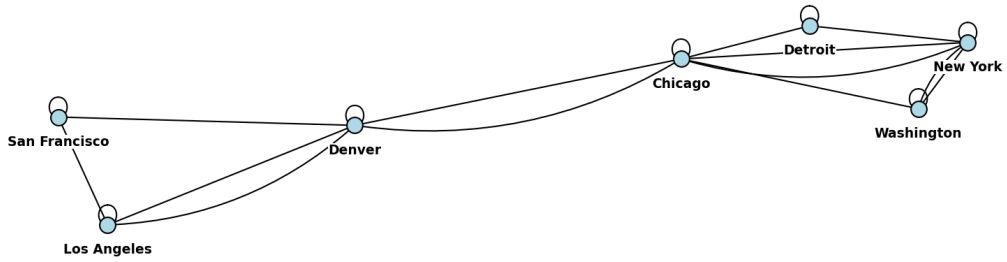


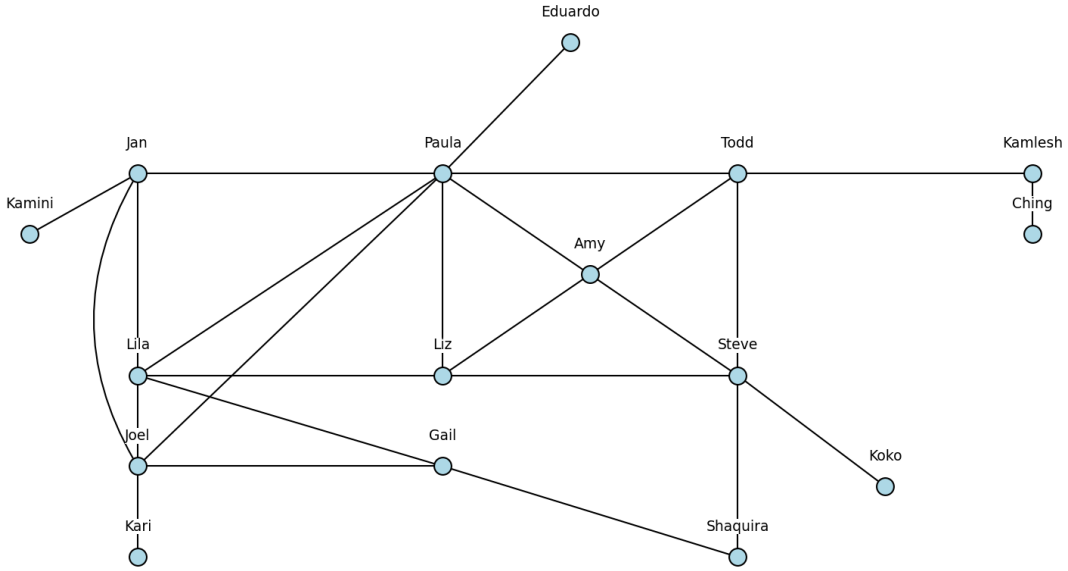
Figure 3 dengan diagnostic link (graf semu)



Gambar 3.2: Figure 1, 2, dan 3 jaringan komputer

Chicago punya derajat terbesar di Figure 1 (4) dan tetap terbesar setelah jalur ganda dan gelang ditambahkan (6, lalu 8). Setiap gelang menambah derajat simpulnya dua, sehingga tujuh gelang di Figure 3 menaikkan jumlah derajat dari 26 menjadi 40.

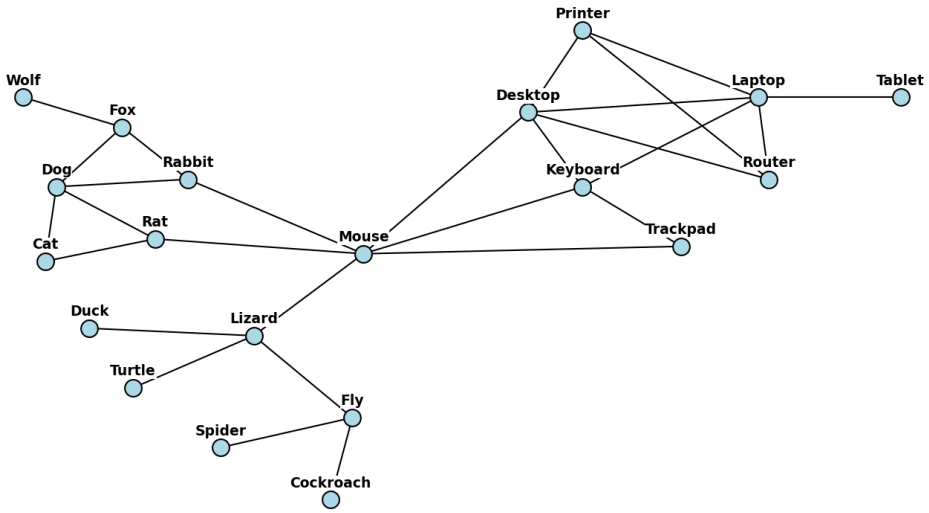
Figure 6 graf pertemanan (acquaintanceship graph)



Gambar 3.3: Figure 6 graf pertemanan

Paula paling banyak kenalan dengan derajat 7, sedangkan Kamini, Eduardo, Koko, Ching, dan Kari masing-masing hanya kenal satu orang.

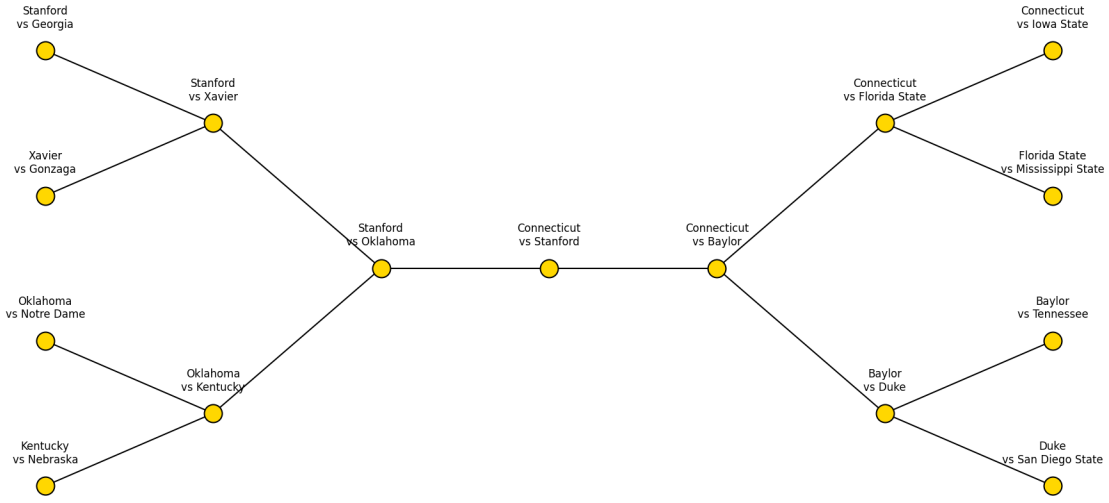
Figure 13 jaringan semantik kata "mouse"



Gambar 3.4: Figure 13 jaringan semantik kata "mouse"

Mouse berderajat 6 dan menjadi penghubung antara kelompok hewan dan kelompok perangkat komputer, sesuai makna ganda kata mouse.

Figure 15 turnamen sistem gugur (pemenang ditulis di depan)



Gambar 3.5: Figure 15 turnamen sistem gugur

Graf turnamen punya 15 simpul dan 14 sisi, terhubung, dan `nx.is_tree()` bernilai True. Pertandingan babak pertama berderajat 1, sedangkan final berderajat 2 karena hanya menerima

pemenang dari dua semifinal.

3.5 Lesson Learnt Tugas

Bagian paling lama justru membaca gambar, bukan menulis kode. Di Figure 2, dua jalur Chicago-New York digambar sangat rapat sampai hampir terlihat seperti satu garis, dan baru jelas ada dua sisi setelah gambar diperbesar. Salah membaca satu sisi langsung mengubah derajat dua simpul sekaligus.

Pada Figure 6, sisi Jan-Joel segaris dengan Jan-Lila dan Lila-Joel. Kalau digambar lurus, sisi itu tertutup dua sisi lain dan terlihat seperti tidak ada, jadi Jan-Joel digambar melengkung seperti di buku. Label nama juga diberi latar putih supaya tetap terbaca walaupun tertimpa garis.

Figure 15 menunjukkan graf tidak selalu berupa jaringan. Turnamen sistem gugur bisa dimodelkan sebagai pohon, yaitu graf terhubung dengan jumlah sisi tepat satu kurangnya dari jumlah simpul.

BAB IV KESIMPULAN

1. NetworkX menyediakan kelas berbeda untuk graf sederhana, graf ganda, graf semu, graf berarah, dan graf ganda berarah. Kelas yang salah bisa menghilangkan sisi ganda tanpa peringatan.

2. Derajat hasil program pada semua graf cocok dengan hitungan manual, dan jumlah derajat selalu dua kali jumlah sisi, termasuk pada graf yang punya gelang.

3. Contoh kode di modul perlu dicek ulang terhadap definisi graf, karena contoh G2 dan G3 tidak sama dengan gambar dan himpunan sisinya.

4. Masalah nyata seperti jaringan komputer, pertemanan, makna kata, dan turnamen bisa dimodelkan sebagai graf, dan bentuk grafnya langsung memperlihatkan simpul yang paling berpengaruh.